



CHITTAGONG UNIVERSITY OF ENGINEERING & TECHNOLOGY

Department of Computer Science & Engineering

Course No : CSE – 312

Course Title : Computer Networks (Sessional)

Lab Report Title :

**Machine Learning Report
(Malware Detection System)**

Date of Submission : 04.12.2023

Submitted by :

Symom Hossain Shohan (1904048)

Mostak Mahmud Chowdhury (1904055)

Md. Sajid Alam Chowdhury (1904064)

Section : A

Group : A₂

Level : 03

Term : 02

Submitted To :

Dr. Mahfuzulhoq Chowdhury

Associate Professor,

Department of CSE, CUET

Remarks

--

Introduction:

The incessant growth of sophisticated malware threatens the integrity and security of digital systems globally. As traditional defense mechanisms struggle to keep pace with evolving threats, the integration of machine learning stands out as a beacon of hope in fortifying our cyber defenses. Machine learning models offer a dynamic approach to malware detection, capable of learning from vast datasets to identify intricate patterns and anomalies. This lab report delves into the application of diverse machine learning methodologies in the realm of malware detection. Exploring their efficacy, limitations, and potential, this study aims to illuminate the evolving landscape of cybersecurity and contribute to the ongoing pursuit of robust solutions against malicious software.

Objectives:

- To understand various Machine Learning algorithms
- To apply them and perform malware detection in the dataset
- To compare accuracy of various machine learning algorithms

Description:

This lab report presents a comprehensive analysis of various machine learning models applied to a specialized malware detection dataset. Leveraging a collection of cutting-edge algorithms including K-Nearest Neighbors (KNN), Random Forest, Decision Tree, Naive Bayes, Logistic Regression, and Support Vector Machines (SVM), this study aims to elucidate their effectiveness in discerning between benign and malicious software.

The dataset utilized in this investigation comprises a diverse range of features extracted from malware samples, encompassing behavioral patterns, code structures, and file attributes. Each machine learning model is meticulously trained, validated, and evaluated using this rich dataset to explore their respective strengths and weaknesses in malware classification.

Through a systematic examination of these models, this report scrutinizes their predictive capabilities, computational efficiency, robustness to varying types of malware, and adaptability to real-world scenarios. The findings of this study are poised to offer valuable insights into the nuanced performance of distinct machine learning methodologies when tasked with the critical mission of malware detection.

Code Analysis:

First, we imported necessary libraries.

```
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
import pickle
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))
plt.style.use('ggplot')
# You can write up to 20GB to the current directory (/kaggle/working/) that gets preserved as output when you create
# You can also write temporary files to /kaggle/temp/, but they won't be saved outside of the current session
```

Then, we imported the dataset using pandas library of python.

```
data = pd.read_csv('/kaggle/input/pe-files-malwares/dataset_malwares.csv')
```

```
data.head()
```

		Name	e_magic	e_cblp	e_cp	e_crlc	e_cparhdr	e_minalloc	e_maxalloc	e_ss	e_sp	...	SectionMaxChar	SectionMainC
0		VirusShare_a878ba26000edaac5c98eff4432723b3	23117	144	3	0	4	0	65535	0	184	...	3758096608	
1		VirusShare_ef9130570fddc174b312b2047f5f4cf0	23117	144	3	0	4	0	65535	0	184	...	3791650880	
2		VirusShare_ef84cdeba22be72a69b198213dada81a	23117	144	3	0	4	0	65535	0	184	...	3221225536	
3		VirusShare_6bf3608e60ebc16cbcff6ed5467d469e	23117	144	3	0	4	0	65535	0	184	...	3224371328	
4		VirusShare_2cc94d952b2efb13c7d6bbe0dd59d3fb	23117	144	3	0	4	0	65535	0	184	...	3227516992	

There are total of 79 columns and 19611 entries in our dataset.

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19611 entries, 0 to 19610
Data columns (total 79 columns):
#   Column              Non-Null Count  Dtype
---  -
0   Name                 19611 non-null object
1   e_magic              19611 non-null int64
2   e_cblp               19611 non-null int64
3   e_cp                 19611 non-null int64
4   e_crlc               19611 non-null int64
5   e_cparhdr            19611 non-null int64
6   e_minalloc           19611 non-null int64
7   e_maxalloc           19611 non-null int64
8   e_ss                 19611 non-null int64
9   e_sp                 19611 non-null int64
10  e_csum               19611 non-null int64
11  e_ip                 19611 non-null int64
12  e_cs                 19611 non-null int64
13  e_lfarlc             19611 non-null int64
14  e_ovno               19611 non-null int64
15  e_oemid              19611 non-null int64
```

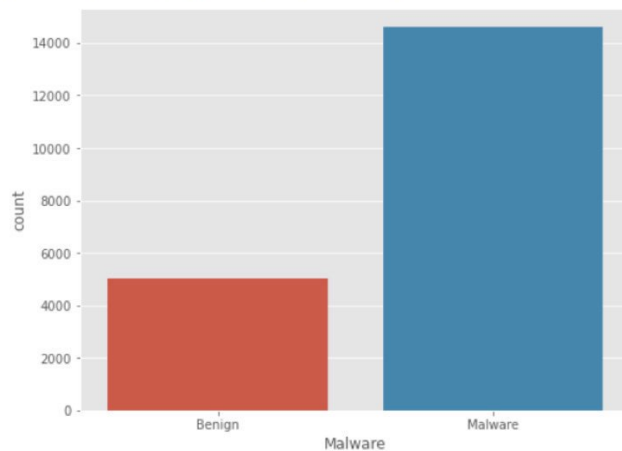
Among them we dropped Name, Machine, TimeDateStamp, and Malware columns.

```
used_data = data.drop(['Name', 'Machine', 'TimeDateStamp', 'Malware'], axis=1)
```

After that we, plotted the total count of malwares, and benign software in a same graph.

```
plt.figure(figsize=(8, 6))
ax=sns.countplot(data['Malware'])
ax.set_xticklabels(['Benign', 'Malware'])
```

```
[Text(0, 0, 'Benign'), Text(0, 0, 'Malware')]
```



Then we split our dataset into testing and training part.

```
X_train, X_test, y_train, y_test = train_test_split(used_data, data['Malware'], test_size=0.2, random_state=0)
```

```
print(f'Number of used features is {X_train.shape[1]}')
```

```
Number of used features is 75
```

We first used the Random Forest Classifier and calculated the accuracy of the model.

```
rfc = RandomForestClassifier(n_estimators=100, random_state=0,
                             oob_score = True,
                             max_depth = 16)
rfc.fit(X_train, y_train)
```

```
RandomForestClassifier(max_depth=16, oob_score=True, random_state=0)
```

```
y_pred = rfc.predict(X_test)
```

Here is the classification report for Random Forest algorithm.

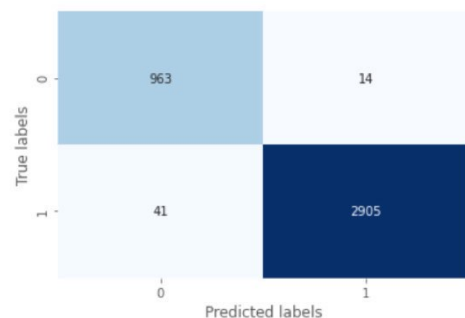
```
print(classification_report(y_test, y_pred, target_names=['Benign', 'Malware']))
```

	precision	recall	f1-score	support
Benign	0.99	0.96	0.97	1004
Malware	0.99	1.00	0.99	2919
accuracy			0.99	3923
macro avg	0.99	0.98	0.98	3923
weighted avg	0.99	0.99	0.99	3923

We also calculated the confusion matrix for Random Forest.

```
ax=sns.heatmap(confusion_matrix(y_pred, y_test), annot=True, fmt="d", cmap=plt.cm.Blues, cbar=False)
ax.set_xlabel('Predicted labels')
ax.set_ylabel('True labels')
```

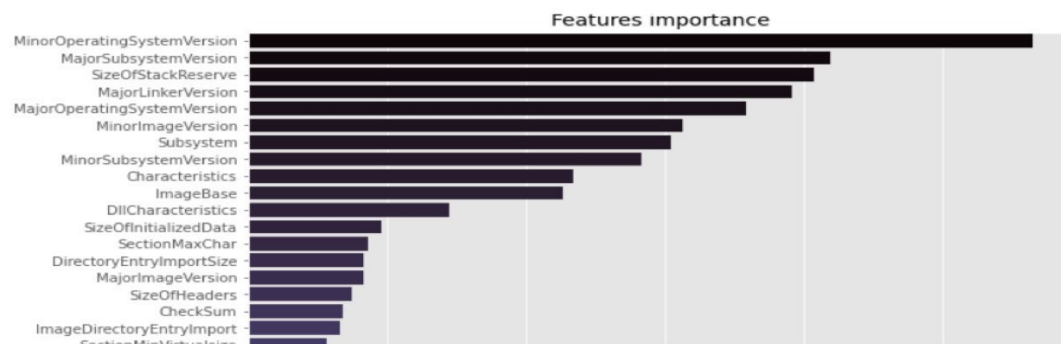
```
Text(33.0, 0.5, 'True labels')
```



Here we can see 41 malware was detected as benign and 14 benign was detected as malware.

After that, we also calculated the feature importance.

```
importance = rfc.feature_importances_
importance_dict = {used_data.columns.values[i]: importance[i] for i in range(len(importance))}
sorted_dict = {k: v for k, v in sorted(importance_dict.items(), key=lambda item: item[1])}
plt.figure(figsize=(10, 20))
sns.barplot(y=list(sorted_dict.keys())[:-1], x=list(sorted_dict.values())[:-1], palette='mako')
plt.title('Features importance')
```



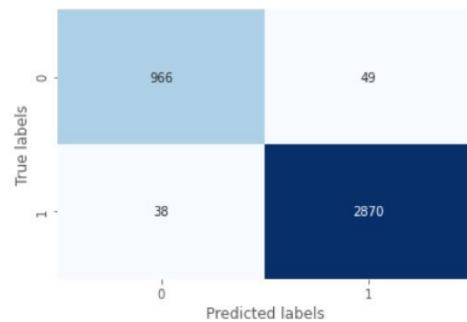
We followed the same procedure for Decision tree. In decision tree the accuracy report was as follows:

```
print(classification_report(y_test, y_pred, target_names=['Benign', 'Malware']))
```

	precision	recall	f1-score	support
Benign	0.95	0.96	0.96	1004
Malware	0.99	0.98	0.99	2919
accuracy			0.98	3923
macro avg	0.97	0.97	0.97	3923
weighted avg	0.98	0.98	0.98	3923

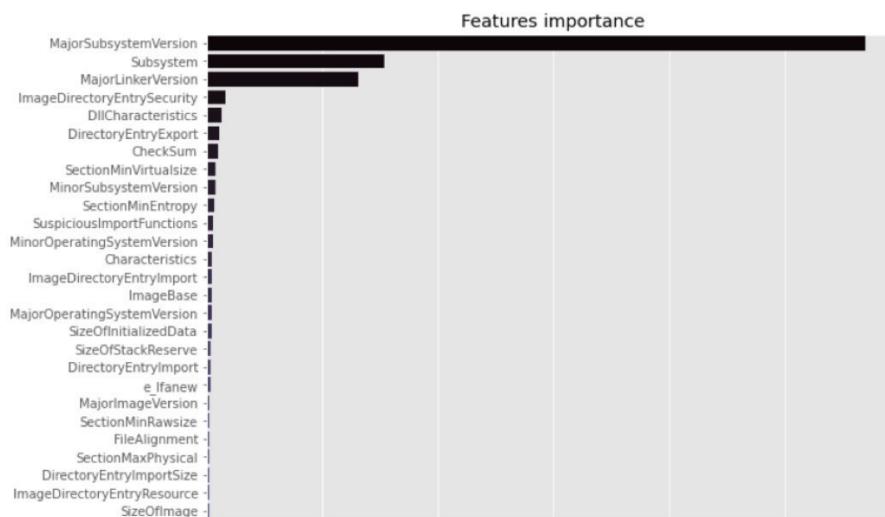
Also, the confusion matrix:

Text(33.0, 0.5, 'True labels')

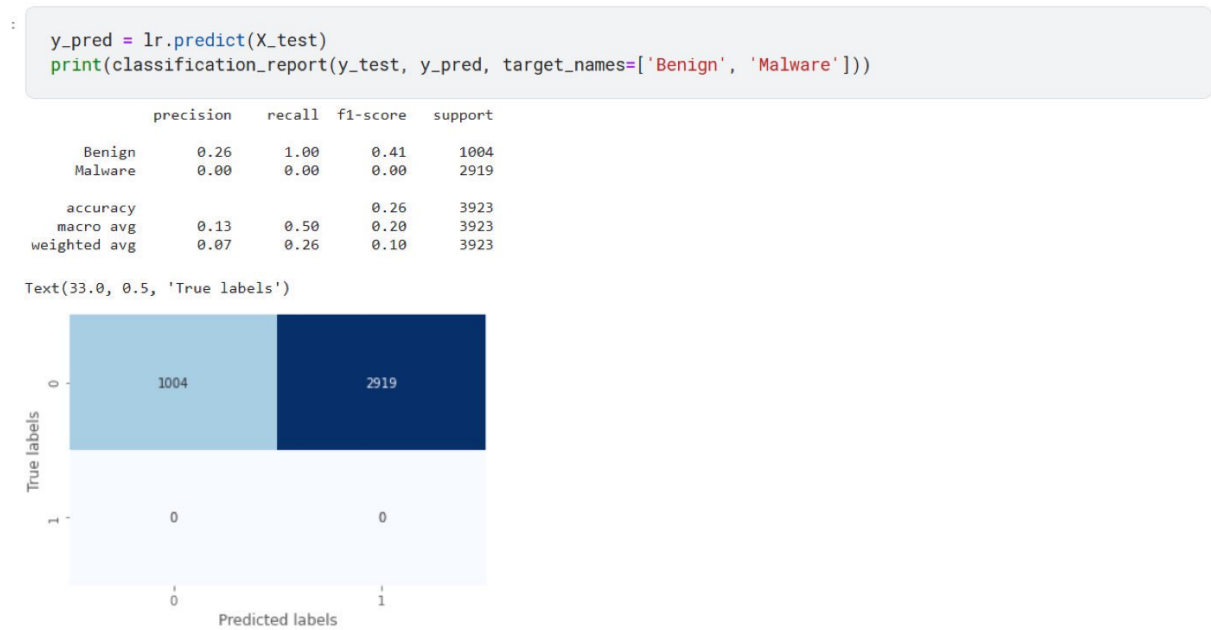


Finally, the feature importance:

Text(0.5, 1.0, 'Features importance')



Now, the classification and confusion matrix for Logistic Regression is given below:

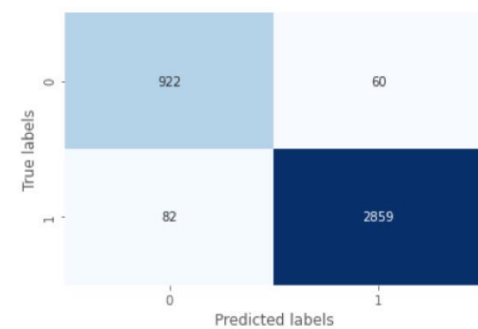


Now, the classification and confusion matrix for SVM is given below:

	precision	recall	f1-score	support
Benign	1.00	0.01	0.03	1004
Malware	0.75	1.00	0.86	2919
accuracy			0.75	3923
macro avg	0.87	0.51	0.44	3923
weighted avg	0.81	0.75	0.64	3923

Finally, for KNN the classification report and confusion matrix is given below:

	precision	recall	f1-score	support
Benign	0.94	0.92	0.93	1004
Malware	0.97	0.98	0.98	2919
accuracy			0.96	3923
macro avg	0.96	0.95	0.95	3923
weighted avg	0.96	0.96	0.96	3923



Discussion:

Following an exhaustive evaluation of various machine learning models for malware detection, this study unequivocally highlights Random Forest as the standout performer among the algorithms investigated. Its exceptional accuracy, resilience to overfitting, and adeptness in discerning intricate patterns position it as a formidable tool in combating evolving cyber threats. While each model showcased unique strengths and limitations, the demonstrated superiority of Random Forest underscores its pivotal role in fortifying cybersecurity. Nonetheless, the dynamic nature of cyber threats necessitates ongoing research and adaptation to ensure robust defenses against emerging malware. Exploring ensemble techniques and hybrid model integrations holds promise for further enhancing the efficacy and adaptability of malware detection systems in the digital landscape.

Reference:

[1] <https://www.kaggle.com/code/maidaly/malware-detection-with-machine-learning>